



Curl Curriculum

Module 1: Basics & GET Requests (1-2 hours)

Core Concepts:

- Default = GET
- Status codes & response headers
- Saving output

Must-Know Commands:

Bash

```
# Simple GET
curl https://httpbin.org/get
```

```
# Show full response (status + headers + body)
curl -v https://httpbin.org/get      # verbose (most useful)
curl -i https://httpbin.org/get      # include headers
```

```
# Only headers (great for quick checks)
curl -I https://httpbin.org/get
```

```
# Save response
curl -o output.json https://httpbin.org/get
curl -O https://example.com/file.pdf  # save with remote filename
```

Key Status Codes to Test:

- 200 OK
- 404 Not Found → `curl -i https://httpbin.org/status/404`
- 500 Internal Server Error → `/status/500`

Practice:

- Hit your own endpoints with different paths.
- Compare `-v`, `-i`, and plain output.

Module 2: Sending Data (POST, PUT, PATCH) (2 hours)

Bash

```
# JSON payload (most common for APIs)
curl -X POST https://httpbin.org/post \
-H "Content-Type: application/json" \
-d '{"name": "Chau", "role": "developer"}
```

```
# Form data (application/x-www-form-urlencoded)
curl -X POST https://httpbin.org/post \
-d "name=Chau&role=developer"
```

```
# File upload
curl -X POST https://httpbin.org/post \
-F "file=@/path/to/image.jpg"
```

Tips:

- Use -X POST (or -X PUT, -X DELETE, etc.)
- H for headers (you'll use this constantly)
- -data-raw or -d for body

Practice:

Test creating resources (expect 201), updating (200/204), and invalid payloads (400).

Module 3: Authentication & Headers (Most Important Module)

Bash

```
# Bearer Token (JWT) - you'll use this daily
curl -H "Authorization: Bearer your-token-here" https://api.yourapp.com/protected
```

```
# Basic Auth
curl -u username:password https://httpbin.org/basic-auth/user/pass
```

```
# Custom headers
curl -H "X-API-Key: abc123" \
-H "Accept: application/json" \
https://...
```

Common Patterns:

- Test 401 Unauthorized → remove token or use wrong one
- Test 403 Forbidden
- Test CORS (check response headers)

Pro Tip: Create a shell alias or function:

Bash

```
alias myapi='curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json"'
```

Module 4: Response Handling & Debugging

Bash

```
# Pretty-print JSON (pipe to jq)
curl https://httpbin.org/get | jq
```

```
# Only get status code
curl -s -o /dev/null -w "%{http_code}" https://httpbin.org/get
```

```
# Write status + time + size to console
curl -w "\nHTTP Code: %{http_code}\nTotal Time: %{time_total}s\n" -o /dev/null -s https://httpbin.org/get
```

Useful Write-Out Variables:

- `%{http_code}`, `%{time_total}`, `%{size_download}`, `%{url_effective}`
-

Module 5: Common Real-World Scenarios

1. **Follow Redirects:** `curl -L https://...`

2. **Cookies/Session:** Bash

```
curl -c cookies.txt https://... # save
curl -b cookies.txt https://... # send
```

3. **Timeouts** (critical for testing): Bash

```
curl --connect-timeout 5 --max-time 10 https://...
```

4. **Ignore SSL (local/dev only):** `curl -k https://self-signed.local`

5. **Proxy** (if needed): `curl -x http://proxy:port ...`

Module 6: Scripting & Automation (Day-to-Day Productivity)

Create a small test script (test-api.sh):

Bash

```
#!/bin/bash
BASE="https://api.yourapp.com"

echo "Testing Health Check..."
curl -s -w "%{http_code}" -o /dev/null "$BASE/health" | grep -q "200" && echo "✅ OK" ||
echo "❌ Failed"

# More tests with assertions...
```

Combine with jq for powerful checks:

Bash

```
curl -s "$BASE/users/1" | jq -e '.id == 1' > /dev/null && echo "ID correct"
```